Công nghệ mã nguồn mở

ThS. Nguyễn Thị Mai Trang

1

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Nội dung

Chương 1 Tổng quan về mã nguồn mở
Chương 2 Hệ điều hành mã nguồn mở
Chương 3 Lập trình SHELL
Chương 4 Lập trình với Python

2

Lập trình với Python

Chương 3

3

Nội dung chương 4

- Giới thiệu
- Các thành phần cơ bản của Python
- Cấu trúc điều khiển
- Hàm
- Đối tượng
- Chuỗi

4

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

4.1. Giới thiệu Python

- Đặc điểm
- Ứng dụng
- Cài đặt
- Công cụ phát triển (IDE)
- Chương trình Python đầu tiên

5

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Giới thiệu Python

- Python là một ngôn ngữ lập trình bậc cao, phổ biến
- Được tạo ra bởi Guido van Rossum, phát hành năm 1991.
- Đặc điểm:
  - Cú pháp đơn giản
  - Ngôn ngữ thông dịch (Interpreted)
  - Đa nền tảng (Cross-platform).
  - Mã nguồn mở (Open Source)
  - Thư viện khổng lồ: thư viện tiêu chuẩn + thư viện của bên thứ ba (ví dụ: NumPy, Pandas, Django)

6

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Ứng dụng của Python

- Phát triển Web (Web Development): các framework như Django và Flask.
- Khoa học Dữ liệu & Học máy (Data Science & Machine Learning):
  - Các thư viện như NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch,...
  - Phân tích dữ liệu, xây dựng mô hình trí tuệ nhân tạo.
- Tự động hóa & Scripting (Automation & Scripting):
  - Dễ dàng viết các script tự động hóa các tác vụ lặp đi lặp lại
  - Ví dụ: xử lý file, gửi email, hoặc tự động hóa các thao tác trên máy tính.
- Phát triển Game: các thư viện như Pygame tạo ra các game 2D
- Lập trình nhúng (Embedded Systems):
  - MicroPython, CircuitPython
  - Lập trình cho các vi điều khiển và thiết bị IoT.

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Cài đặt Python

- Tải Python phiên bản mới nhất từ trang chủ python.org.
- Thực hiện cài đặt: lưu ý chọn "Add Python to PATH“
- Kiểm tra cài đặt:
  - Mở Command Prompt (Windows)/ Terminal (macOS/Linux)
  - Gõ lệnh:

    py --version
  - Kiểm tra trình quản lý gói pip:

    pip --version

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

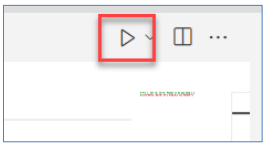
Công cụ phát triển (IDE)

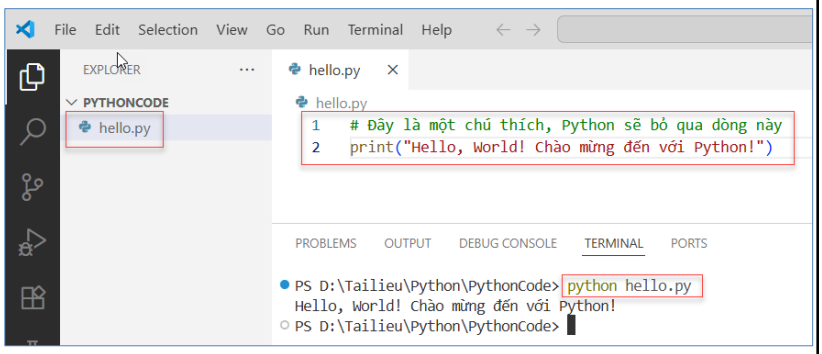
- Visual Studio Code (VS Code):
  - Cài Extensions “Python”
- PyCharm:
  - Là một IDE chuyên dụng cho Python được phát triển bởi JetBrains.
  - Phiên bản Community (miễn phí) và Professional (trả phí)
- Jupyter Notebook:
  - Công cụ dung cho khoa học dữ liệu

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Chương trình Python đầu tiên

- Tạo, lưu file hello.py
- Thực thi:
  - python hello.py
  - Hoặc click như hình





TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

4.2 Các thành phần cơ bản của Python

- **Biến**
- **Kiểu dữ liệu**
- **Toán tử**
- **Nhập/Xuất dữ**

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Biến

- Tạo biến:
  - sử dụng phép gán
  - Kiểu dữ liệu của biến tùy theo giá trị gán vào

```
message = "Hello Python world!"
number = 10
```

- Quy tắc đặt tên biến:
  - Tên biến chỉ gồm chữ cái, chữ số và dấu gạch dưới, không chứa khoảng trắng
  - Bắt đầu bằng chữ cái hoặc dấu gạch dưới.
  - Không trùng với từ khóa
  - Nên ngắn gọn nhưng có nghĩa

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Kiểu dữ liệu

- Số nguyên (int)
- Số thực (float)
- Chuỗi ký tự (str):
  - Đặt trong dấu nháy đơn (‘)/nháy kép (")
- Logic: True/ False
- Kiểm tra kiểu dữ liệu của biến:
  - Dùng hàm type()

```
message = "Hello Python world!"
user_name = 'maria'
number = 10
price = 15.5
is_logged_in = True
has_permission = False
print(type(user_name))
print(type(number))
print(type(price))
print(type(is_logged_in))
```

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Kiểu dữ liệu

- **Ép kiểu (Type Casting):**
  - Cú pháp:
    - datatype(variable): chuyển về kiểu datatype
    - eval(exp): chuyển kiểu tự động dựa vào kết quả exp
  - Ví dụ:
    - int(“100”)
    - float(“15.9”)
    - eval(input(“Enter a number: ”)

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Toán tử (Operators)

- **Toán tử số học (Arithmetic Operators)**

Toán tử	Mô tả	Ví dụ	Kết quả
+	cộng	10 + 5	15
-	trừ	10 - 5	5
*	nhân	10 * 5	50
/	chia (trả về kiểu float)	10 / 4	2.5
//	chia lấy phần nguyên	10 // 4	2
%	chia lấy phần dư	10 % 3	1
**	lũy thừa	2 ** 3	8

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Toán tử (Operators)

- **Toán tử so sánh (Comparison Operators)**

Toán tử	Mô tả	Ví dụ	Kết quả
==	Bằng	5 == 5	True
!=	Không bằng	5 != 5	False
>	Lớn hơn	10 > 5	True
<	Nhỏ hơn	10 < 5	False
>=	Lớn hơn hoặc bằng	5 >= 5	True
<=	Nhỏ hơn hoặc bằng	10 <= 5	False

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Toán tử (Operators)

- **Toán tử logic (Logical Operators)**

e1	e2	e1 and e2	e1 or e2	not e1	not e2
False	False	False	False	True	True
False	True	False	True	True	False
True	False	False	True	False	True
True	True	True	True	False	False

Ví dụ

(5 > 3) and (10 < 20) -> True

(5 > 10) or (10 < 20) -> True

not (5 > 3) -> False

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Nhập và Xuất

- **Xuất:**

```
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

– objects: các đối tượng được in

– sep: các ký tự phân cách, mặc định là một khoảng trắng.

– end: giá trị cuối cùng được in ra màn hình, mặc định là ‘\n’.

– file: mặc định hàm print sẽ ghi nội dung vào file sys.stdout

– flush: mặc định là False.

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Nhập và Xuất

- **Xuất:**

Code	Output
`print("Today is Monday.")` `print("I like string beans.")`	`Today is Monday.` `I like string beans.`
`print("Today", "is", "Monday")` `print("Today", "is", "Monday", sep="...")`	`Today is Monday` `Today...is...Monday`
`print("Today is Monday, ", end="")` `print("I like string beans.")`	`Today is Monday, I like string beans.`
`print("Today", "is", "Monday", sep="? ", end="!!")` `print("I like string beans.")`	`Today? is? Monday!!I like string beans.`

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Nhập và Xuất

- **Định dạng xuất với toán tử %**

Format	Ý nghĩa	Ví dụ
%s	Chuỗi (string)	print("Tên tôi là %s" % "Thanh") # Tên tôi là Thanh
%d / %i	Số nguyên (integer)	print("Tôi có %d cuốn sách" % 5) # Tôi có 5 cuốn sách
%f	Số thực (float)	print("PI = %f" % 3.14159) # PI = 3.141590
%.2f	Số thực với số chữ số thập phân cố định	print("Pi = %.2f" % 3.14159) # Pi = 3.14
%e / %E	Số thực dạng khoa học (exponential)	print(" %e" % 0.0000123)#1.230000e-05
%c	Ký tự (character)	print("Ký tự: %c" % 65) # Ký tự: A
%o	Số bát phân (octal)	print("Octal: %o" % 10) # Octal: 12
%x / %X	Số thập lục phân (hex, thường/hoa)	print("Hex: %x" % 255) # Hex: ff

print("Tên: %s, Tuổi: %d, Điểm: %.1f" % ("Thanh", 20, 8.75))
 # Kết quả: Tên: Thanh, Tuổi: 20, Điểm: 8.8

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Nhập và Xuất

- **Định dạng xuất với hàm format**

```
price = 49.9999
product = "Book"
print("The price of the {0} is ${1:.2f}".format(product, price))
# Output: The price of the Book is $50.00
```

- **Định dạng xuất với các hàm str**

```
cstr = "I love python programming"
print("Center aligned: ")
print(cstr.center(40, '#'))
print("left aligned: ")
print(cstr.ljust(40, '-'))
print("right aligned: ")
print(cstr.rjust(40, ' '))
```

```
Center aligned:
#######I love python programming########
left aligned:
I love python programming---------------
right aligned:
                I love python programming
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Nhập và Xuất

- **Xuất:** Ví dụ
  - In chuỗi, giá trị của biến

```
product_name = "Laptop Dell latitute"
price = 500
print("Product name:", product_name, "Price:", price)
```

  - Dùng f-string: đặt ký tự f trước dấu nháy kép, tên biến trong { }

```
product_name = "Laptop Dell latitute"
price = 500
print(f"Product name: {product_name} Price: {price}")
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Nhập và Xuất

- **Nhập dữ liệu: input()**
  - Nhận dữ liệu do người dùng nhập vào từ bàn phím.
  - Hàm input() luôn trả về một chuỗi (str)

```
user_name = input("Nhập tên: ")
print(f"Xin chào, {user_name}!")
user_age_str = input("Nhập tuổi: ")
print(f"Kiểu dữ liệu: {type(user_age_str)}")
```

→

```
Nhập tên: Thanh
Xin chào, Thanh!
Nhập tuổi: 25
Kiểu dữ liệu: <class 'str'>
```

```
ten_sp = input("Tên sản phẩm: ")
gia_mua = float(input("Gía mua: "))
gia_ban = float(input("Giá bán: "))
print(f"Sản phẩm {ten_sp} lãi là {gia_ban - gia_mua}")
```

→

```
Tên sản phẩm: Lap top
Gía mua: 500
Giá bán: 550
Sản phẩm Lap top lãi là 50.0
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Nhập và Xuất

- **Nhập dữ liệu: input()**
  - Lưu ý: cần chuyển kiểu khi tính toán

```
ten_sp = input("Tên sản phẩm: ")
gia_mua = float(input("Gía mua: "))
gia_ban = float(input("Giá bán: "))
print(f"Sản phẩm {ten_sp} lãi là {gia_ban - gia_mua}")
```

→

```
Tên sản phẩm: Lap top
Gía mua: 500
Giá bán: 550
Sản phẩm Lap top lãi là 50.0
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Chú thích

- Chú thích một dòng (Single-line comment): Bắt đầu bằng dấu thăng (#).
- Chú thích nhiều dòng (Multi-line comment): Đặt nội dung chú thích bên trong cặp ba dấu nháy kép (""") hoặc ba dấu nháy đơn (''').

```
# Say hello to everyone.
print("Hello Python people!")
"""Say hello
   to everyone"""
print("Hello Python people!")
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

4.3 Cấu trúc điều khiển

- **Cấu trúc if-else**
  - Cú pháp:

```
if condition :
        if block
else:
        else block
```

```
if condition1 :
        if block1
elif conditon2:
        if block2
else:
        else block
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Cấu trúc điều khiển

- **Cấu trúc if-else** – Ví dụ:

```
num1 = int(input("Số thứ 1: "))
num2 = int(input("Số thứ 2: "))
if num1 == num2:
    print("Hai số bằng nhau")
elif num1 > num2:
    print("Số thứ nhất lớn hơn số thứ hai")
else:
    print("Số thứ nhất nhỏ hơn số thứ hai")
```

```
value = eval(input("Nhập một số từ 0 đến 10: "))
if value >= 0: # First check
    if value <= 10: # Second check
        print(value, "hợp lệ")
    else:
        print(value, "lớn hơn phạm vi yêu cầu")
else:
    print(value, "nhỏ hơn phạm vi yêu cầu")
print("Done")
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Cấu trúc điều khiển

- **Biểu thức điều kiện**

*expression*1 if *condition* else *expression*2

```
a = int(input("Nhập số thứ nhất: "))
b = int(input("Nhập số thứ hai: "))
max = a if a > b else b
print(f"Số lớn hơn là {max}")
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Cấu trúc điều khiển

- **Cấu trúc match**

```
match expression:
  case x:
    code block
  case y:
    code block
  case z:
    code block
  case _:
    code block
```

```
day = 7 #Kết quả là Sunday
match day:
  case 1:
    print("Monday")
  case 2:
    print("Tuesday")
  case 3:
    print("Wednesday")
  case 4:
    print("Thursday")
  case 5:
    print("Friday")
  case 6:
    print("Saturday")
  case 7:
    print("Sunday")
  case _:
    print("Invalid")
```

```
day = 0 #Kết quả là Invalid
match day:
  case 1:
    print("Monday")
  case 2:
    print("Tuesday")
  case 3:
    print("Wednesday")
  case 4:
    print("Thursday")
  case 5:
    print("Friday")
  case 6:
    print("Saturday")
  case 7:
    print("Sunday")
  case _:
    print("Invalid")
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Cấu trúc điều khiển

- **Vòng lặp while**

```
while condition :
    block
```

```
'''tính tổng các số lớn hơn 0 nhập từ bàn phím
Nhập 0 để kết thúc'''
x = 1
sum = 0
while x > 0:
    x = int(input("Nhập số nguyên lớn hơn 0, nhập 0 để dừng: "))
    if x > 0:
        sum += x
print(f"Tổng các số đã nhập là {sum}")
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Cấu trúc điều khiển

- **Vòng lặp for**

for variable in range(begin,end,step)
block

  – begin: giá trị khởi tạo của variable
  – end: giá trị làm điều kiện kết thúc vòng lặp
  – step: giá trị cập nhật biến đếm (mặc định là +1)

```
#tính tổng các số từ 1 đến 10
sum = 0
for i in range (1,11):
    sum += i
```

```
sum = 0
for i in range(10, 0, -1):
    sum += i
print("Tổng các số từ 10 đến 1 là: {0}".format(sum))
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Cấu trúc điều khiển

- **Lệnh break**:
  – Thoát khỏi vòng lặp

```
sum = 0
for i in range(1,11):
    if(i % 5 == 0):
        break;
    sum = sum + i
```

sum = 10

- **Lệnh continue**:
  – Bỏ qua các lệnh còn lại trong vòng lặp
  – Tiếp tục thực hiện vòng lặp kế tiếp

```
sum = 0
for i in range(1,11):
    if(i % 2 == 0):
        continue;
    sum = sum + i
```

sum = 25

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

4.5 Hàm

- Các hàm thư viện math
- Các hàm thư viện time
- Hàm người dùng

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm thư viện math

- **Các hằng số trong math**
  - math.pi: pi ≈ 3.14159
  - math.e: e ≈ 2.71828
  - math.tau: 2*pi
  - math.inf: vô cực
  - math.nan: Not a Number

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm thư viện math

- **Các hàm số học cơ bản**
  - ceil(x): làm tròn lên
  - floor(x): làm tròn xuống
  - trunc(x): bỏ phần thập phân
  - copysign(x,y): trị tuyệt đối của x theo dấu y
  - fabs(x): trị tuyệt đối (float)
  - fmod(x,y): phần dư phép chia nguyên
  - gcd(a,b): ước chung lớn nhất
  - lcm(a,b): bội chung nhỏ nhất
  - isfinite(x), isinf(x), isnan(x): kiểm tra x có phải là số hữu hạn, vô hạn, hay không phải là số

```
import math
print(math.ceil(4.2), math.floor(4.8), math.trunc(-3.9))
print(math.gcd(12,18), math.lcm(12,18))
```

```
5 4 -3
6 36
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm thư viện math

- **Hàm mũ, logarit, lũy thừa, căn**
  - pow(x,y): x^y (float)
  - exp(x): e^x
  - log(x[,base]): logarit trên cơ số base
  - log2(x), log10(x)
  - math.sqrt(x): căn bậc 2 của x
  - math.isqrt(n): căn nguyên của n

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm thư viện math

- **Các hàm số học đặc biệt**
  - prod(iterable, *, start=1): tích các phần tử
    - iterable: dãy số (list, tuple, …).
    - start: tùy chọn, mặc định = 1, giá trị khởi tạo để nhân với tất cả phần tử.
  - factorial(n): giai thừa của n
  - math.comb(n, k): số tổ hợp
  - perm(n,k): số chỉnh hợp

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm thư viện time

- **struct_time**: cấu trúc và các thuộc tính
  - **tm_year**: năm đầy đủ (ví dụ: 2025)
  - **tm_mon**: tháng trong năm (1–12)
  - **tm_mday**: ngày trong tháng (1–31).
  - **tm_hour**: giờ trong ngày (0–23).
  - **tm_min**: phút trong giờ (0–59)
  - **tm_sec**: giây trong phút (0–59)
  - **tm_wday**: thứ trong tuần (0:Monday – 6:Sunday)
  - **tm_yday**: Ngày thứ bao nhiêu trong năm (1–366).
  - **tm_isdst**: Cờ Daylight Saving Time (giờ mùa hè).
    - 0/1/-1: không/đang áp dụng DST
    - -1: không biết / hệ thống tự xác định.

```
Year: 2025
Month: 8
Day: 27
Hour: 11
Minute: 34
Second: 45
Weekday: 2
Yearday: 239
DST: 0
```

D:\Tailieu\Python\PythonCode\structTime.

```
st = time.localtime()
print("Year:", st.tm_year)
print("Month:", st.tm_mon)
print("Day:", st.tm_mday)
print("Hour:", st.tm_hour)
print("Minute:", st.tm_min)
print("Second:", st.tm_sec)
print("Weekday:", st.tm_wday)
print("Yearday:", st.tm_yday)
print("DST:", st.tm_isdst)
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm thư viện time

- **time()/time_ns()**: trả về thời gian hiện tại tính bằng s/ns, kiểu float/int, kể từ 01/01/1970
- **ctime([secs])**: trả về chuỗi dạng ctime.
  – secs: float/int, tùy chọn, mặc định là time().

```
import time
print(time.time())
print(time.ctime())
print(time.ctime(0))
```

```
1756268080.912507
Wed Aug 27 11:14:40 2025
Thu Jan  1 07:00:00 1970
```

https://docs.python.org/3/library/time.html

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm thư viện time

- **localtime([secs])**: trả về struct_time từ số giây (float/int)
- **gmtime([secs])**: trả về struct_time theo UTC

```
print("localtime--------------")
st = time.localtime()
print(st.tm_year, st.tm_mon, st.tm_mday)
print(st.tm_hour, st.tm_min, st.tm_sec)
print(st.tm_wday, st.tm_yday, st.tm_isdst)
print("gmtime--------------")
st = time.gmtime()
print(st.tm_year, st.tm_hour)
```

```
localtime--------------
2025 8 27
11 23 23
2 239 0
gmtime--------------
2025 4
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm thư viện time

- **strptime(string, fmt)**: trả về struct_time từ chuỗi fmt (str)
- **mktime(st)**: st kiểu struct_time → giây (float)
- **strftime(fmt, t)**: chuyển struct_time t → chuỗi fmt (str)

```
print("strptime--------------")
parsed = time.strptime('2025-09-01 08:15:00','%Y-%m-%d %H:%M:%S')
print(parsed.tm_year, parsed.tm_hour)
print("strftime--------------")
s = time.strftime('%Y-%m-%d %H:%M:%S', time.localtime())
print(s)
print("mktime--------------")
st = time.localtime()
ts = time.mktime(st)
print(ts)
```

```
strptime--------------
2025 8
strftime--------------
2025-08-27 16:09:50
mktime--------------
1756285790.0
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm thư viện time

- **perf_counter()**: trả về khoảng thời gian đã trôi qua tính bằng giây từ khi hàm này được gọi

```
import time
print("Nhập tên bạn: ", end="")
start_time =  time.perf_counter()
name = input()
elapsed = time.perf_counter() - start_time
print("Thời gian trả lời của bạn là: %.3f"% elapsed,  "giây" )
```

```
Nhập tên bạn: Trang
Thời gian trả lời của bạn là: 8.478 giây
```

- sleep(secs): tạm dừng chương trình trong secs giây.

```
print('Start...');
time.sleep(1.5);
print('...End')
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Hàm người dùng

- Định nghĩa hàm: từ khóa **def**
  - Cú pháp:

```
def ten_ham(tham_so):
    # Khối lệnh
    return gia_tri
```

  - Hàm có thể trả về một hoặc nhiều giá trị bằng lệnh return.
  - Nếu không có return, hàm sẽ trả về None.
  - Ví dụ:

```
def cong(a, b):
    return a + b
print(cong(3, 5))  # Kết quả: 8
print(cong(10, 20))  # Kết quả: 30
```

```
def calculation(a, b):
    return a + b, a - b
add, sub = calculation(40, 10)
print(add, sub)
# Kết quả: 50 30
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Hàm người dùng

- **Truyền tham số cho hàm**
  - Hàm có thể nhận tham số để xử lý dữ liệu đầu vào.
  - Khi gọi hàm, giá trị truyền vào gọi là đối số.
  - Ví dụ:

```
def cong(a, b):
    return a + b
print("Đây là hàm cộng hai số:")
a = int(input("Nhập số thứ nhất: "))
b = int(input("Nhập số thứ hai: "))
print(f"{a} + {b} = {cong(a, b)}")
```

```
Đây là hàm cộng hai số:
Nhập số thứ nhất: 10
Nhập số thứ hai: 5
10 + 5 = 15
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Hàm người dùng

- **Tham số ngầm định**
  - Có thể định nghĩa giá trị mặc định cho tham số nếu không truyền đối số khi gọi hàm.

```
def cong(a, b = 1):
    return a + b
a = int(input("Nhập số thứ nhất: "))
print(cong(a)) # in ra a + 1
```

```
def cong(a = 5, b = 1):
    return a + b
print(cong()) # 6
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Hàm người dùng

- **Tham số tùy biến *args và **kwargs**
  - *args: cho phép truyền vào nhiều đối số dạng tuple.
  - **kwargs: cho phép truyền vào nhiều đối số dạng từ khóa (dict).

```
def tong(*args):
    return sum(args)
print(tong(1, 2, 3, 4))  # Kết quả: 10
def in_thong_tin(**kwargs):
    for key, value in kwargs.items():
        print(f'{key}: {value}')
in_thong_tin(ten='Thanh', tuoi=20)
```

```
10
ten: Thanh
tuoi: 20
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Hàm người dùng

- **Biến cục bộ và toàn cục**
  - Biến cục bộ: khai báo trong hàm, chỉ truy xuất trong hàm.
  - Biến toàn cục: khai báo ngoài hàm, có thể truy xuất trong toàn bộ chương trình.

```
x = 10
def test():
    y = 5
    print('Bên trong hàm:', x + y)
test() # Kết quả: 15
print('Bên ngoài hàm:', x)  # Kết quả: 10
print('Bên ngoài hàm:', y)  # NameError: name 'y' is not defined
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Hàm người dùng

- Hàm **lambda**:
  - Cú pháp: **lambda [params]: expression**
  - params: danh sách các tham số
  - expression: biểu thức, kết quả là giá trị trả về của hàm
  - Ví dụ:

```
hello = lambda : "Hello World!"
print(hello())   # Kết quả: Hello World!
```

```
tong = lambda a, b, c: a + b + c
print(tong(2, 3, 4))   # Kết quả: 9
```

```
maxValue = lambda a, b: a if a > b else b
print(maxValue(7, 3))   # Kết quả: 7
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Hàm người dùng

- **Hàm lồng nhau (Nested Function)**
  - Python cho phép định nghĩa hàm bên trong một hàm khác.

```
def test():
    def test1():
        return 'code in test1'
    return test1()
print(test())  # Kết quả: code in test1
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Hàm người dùng

- **Gán tên khác cho hàm**
  - Python cho phép gán một tên khác cho hàm đã có và sử dụng như hàm gốc

```
def display_student(name, age):
    print(name, age)
display_student("Thanh", 26)
showStudent = display_student
showStudent("Hoa", 20)
# Output: Thanh 26
# Output: Hoa 20
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm bậc cao

- **map()**: thay thế cho vòng lặp for của Python
  - Cú pháp: map(function name, iterable(list of items))
  - Cho phép áp dụng cùng một hàm cho tất cả các mục được truyền làm đối số cho hàm map()
  - Khi áp dụng hàm map() cho danh sách, nó sẽ chuyển đổi thành một tập hợp các phần tử mới được gọi là iterator. Các phần tử được truyền vào làm đối số được gọi là iterable .

```
def add_num(y):
  return y+y
arrNumber = (2,4,6,8,10)
result = map(add_num, arrNumber)
print(list(result))
# Output: [4, 8, 12, 16, 20]
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm bậc cao

- **map()**: đơn giản hóa vòng lặp for, có thể kết hợp với hàm lambda

map(function_name,iterable_1,iterable_2,iterable_3,...)

```
def add_num(a,b,c):
  return a + b + c
arr1= (2,4,6,8,10)
arr2= (3,6,9,12,15)
arr3= (4,8,12,16,20)
result = map(add_num, arr1, arr2, arr3)
print(list(result))
# output: [9, 18, 27, 36, 45]
```

```
arr1= (2,4,6,8,10)
arr2= (3,6,9,12,15)
arr3= (4,8,12,16,20)
result = map(lambda x, y, z: x + y + z, arr1, arr2, arr3)
print(list(result))
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm bậc cao

- **filter()**: filter(function name, iterable(list of items))
  - lọc các phần tử theo điều kiện
  - Có thể kết hợp với hàm lambda

```
def odd(numbers):
    return numbers%2 == 0
sequence = [1,2,3,4,5,6,7,8,9,10]
filtered = filter(odd, sequence)
print('The filtered numbers are: ')
for s in filtered:
        print(s, end=' ')
# Output: 2 4 6 8 10
```

```
sequence = [1,2,3,4,5,6,7,8,9,10]
filtered = filter(lambda x: x%2 == 0, sequence)
for s in filtered:
        print(s, end=' ')
# Output: 2 4 6 8 10
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Các hàm bậc cao

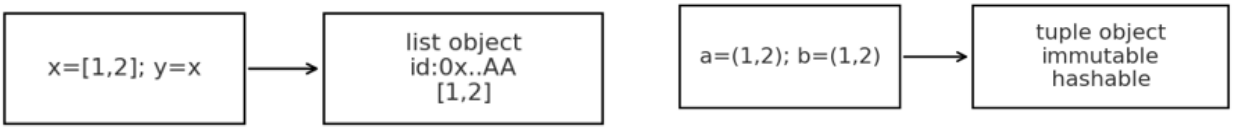
- **all()**: all(iterable)
  - iterable: [condition_1, condition_2, ...]
  - Trả về True nếu tất cả condition_i là True
- **any()** any(iterable)
  - Trả về True nếu có một condition_i là True
- Quy ước:
  all([ ]) là True, any([ ]) là False

```
diem = [7, 8, 6, 5, 9]
print(all(d >= 5 for d in diem))  # True
print(any(d < 5 for d in diem))   # False
tuoi = [17, 18, 26, 25, 19]
print(all(t >= 18 for t in tuoi))  # False
print(any(t < 18 for t in tuoi))   # True
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

4.5 Lớp - Đối tượng

- Lớp (class): là khuôn mẫu mô tả các thuộc tính (dữ liệu) và phương thức (hành vi).
- Đối tượng (object): là thể hiện cụ thể của lớp
- Mọi thứ trong Python đều là object.
- Thuộc tính: biến lưu trữ thông tin của đối tượng
- Phương thức: hàm bên trong lớp, mô tả hành vi của đối tượng



TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Lớp - Đối tượng

- Phương thức khởi tạo

```
class HinhTron:
    def __init__(self, ban_kinh):
        self.ban_kinh = ban_kinh

    def dien_tich(self):
        return 3.14 * self.ban_kinh * self.ban_kinh

ht = HinhTron(5)
print(ht.dien_tich())   # Kết quả: 78.5
```

Phương thức khởi tạo

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Lớp - Đối tượng

- In đối tượng dạng chuỗi: __str__()

```
class Student:
    def __init__(self, name, sid):
        self.name = name
        self.sid = sid
s = Student("Thanh", "S001")
print(s)
# Kết quả: <__main__.Student object at 0x...>
```

```
class Student:
    def __init__(self, name, sid):
        self.name = name
        self.sid = sid
    def __str__(self):
        return f"Student {self.name} (ID: {self.sid})"
s = Student("Thanh", "S001")
print(str(s))     # gọi __str__
print(s)          # print() cũng gọi __str__
```

```
Student Thanh (ID: S001)
Student Thanh (ID: S001)
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Lớp - Đối tượng

- Tham số **self**:
  - Tham chiếu đến thể hiện hiện tại của lớp (tương tự this của các NNLT khác), dùng để truy cập các biến thuộc về lớp đó.
  - Tham số này có thể đặt tên khác, nhưng phải là tham số đầu tiên của bất kỳ hàm nào trong lớp

```
class Person:
  def __init__(myobj, name, age):
    myobj.name = name
    myobj.age = age

  def myfunc(self):
    print(f"Tên tôi là {self.name} và tôi {self.age} tuổi")
p1 = Person("Thanh", 21)
p1.myfunc()
# Kết quả: Tên tôi là Thanh và tôi 21 tuổi
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Lớp - Đối tượng

- **Cập nhật/xóa thuộc tính của đối tượng**

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age
  def myfunc(self):
    print("Hello my name is " + self.name)
p1 = Person("John", 36)
p1.age = 40
print(p1.age)#kết quả 40
```

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age
  def myfunc(self):
    print("Hello my name is " + self.name)
p1 = Person("John", 36)
del p1.age
print(p1.age)
```

```
Traceback (most recent call last):
  File "demo_class7.py", line 13, in <module>
    print(p1.age)
AttributeError: 'Person' object has no attribute 'age'
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Lớp - Đối tượng

- **Kế thừa**

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname
  def printname(self):
    print(self.firstname, self.lastname)
class Student(Person):
  pass
x = Student("Mike", "Olsen")
x.printname() # kết quả: Mike Olsen
```

– Câu lệnh pass: được dùng khi khai báo một lớp rỗng

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Lớp - Đối tượng

- **Phương thức khởi tạo trong lớp kế thừa**
  - Được dùng để ghi đè hàm __init__() của lớp cha

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname
  def printname(self):
    print(self.firstname, self.lastname)
class Student(Person):
  def __init__(self, fname, lname):
    Person.__init__(self, fname, lname)
x = Student("Mike", "Olsen")
x.printname() # Kết quả: Mike Olsen
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Lớp - Đối tượng

- **Hàm super()**:
  - Cho phép lớp con kế thừa tất cả các phương thức và thuộc tính từ lớp cha mà không cần sử dụng tên lớp cha

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname
  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)

x = Student("Mike", "Olsen")
x.printname() #Kết quả: Mike Olsen
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Lớp - Đối tượng

- **Thêm thuộc tính và phương thức trong lớp kế thừa():**

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname
  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year
  def welcome(self):
    print("Welcome", self.firstname, self.lastname, "to the class of", self.graduationyear)

x = Student("Mike", "Olsen", 2024)
x.welcome() #Welcome Mike Olsen to the class of 2024
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

Lớp - Đối tượng

- **Đa hình (polymorphism):**

```
class Vehicle:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model
  def move(self):
    print("Move!")
```

```
class Car(Vehicle):
  pass
class Boat(Vehicle):
  def move(self):
    print("Sail!")
class Plane(Vehicle):
  def move(self):
    print("Fly!")
car1 = Car("Ford", "Mustang") #Create a Car object
boat1 = Boat("Ibiza", "Touring 20") #Create a Boat object
plane1 = Plane("Boeing", "747") #Create a Plane object
for x in (car1, boat1, plane1):
  print(x.brand)
  print(x.model)
  x.move()
```

```
Ford
Mustang
Move!
Ibiza
Touring 20
Sail!
Boeing
747
Fly!
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

4.6 List

- Python có bốn kiểu dữ liệu tập hợp cơ bản:
  - **List**: là một tập hợp được sắp xếp và có thể thay đổi, cho phép các phần tử trùng lặp.
  - **Tuple** là một tập hợp được sắp xếp và không thể thay đổi, cho phép các phần tử trùng lặp.
  - **Set** là một tập hợp không được sắp xếp, không thể thay đổi, không được lập chỉ mục, không có phần tử trùng lặp.
  - **Dictionary** là một tập hợp được sắp xếp, có thể thay đổi, không có phần tử trùng lặp.

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

List

- Tạo đối tượng List: dùng **[ ]**
- Truy cập phần tử: dùng chỉ số (index)
  - index >= 0: từ đầu danh sách
  - index < 0: từ cuối danh sách

```
colors = ["red", "green", "blue", "yellow", "white "]
print(colors[0])  # Output: red
print(colors[1 ])  # Output: green
print(colors[-1])  # Output: white
print(colors[-2])  # Output: yellow
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

List

- Có thể truy xuất một dãy phần tử trong list
  - Cú pháp: list_name[start:end]
  - Trả về danh sách các phần tử từ start đến end – 1
  - Nếu bỏ qua start, sẽ lấy từ 0, bỏ qua end: lấy từ start đến cuối
  - end < 0: lấy từ start đến phần tử trước end (tính từ cuối)
  - start <0, end < 0: tính từ cuối danh sách

```
colors = ["red", "green", "blue", "yellow", "white", "black"]
print(colors[2:5]) # Output: ['blue', 'yellow', 'white']
print(colors[:3]) # Output: ['red', 'green', 'blue']
print(colors[3:]) # Output: ['yellow', 'white', 'black']
print(colors[1:-2]) # Output: ['green', 'blue', 'yellow']
print(colors[-4:-1]) # Output: ['blue', 'yellow', 'white']
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

List

- Kiểm tra một phần tử có trong danh sách
  - Cú pháp: if item in list_name:

```
colors = ["red", "green", "blue", "yellow", "white", "black"]
color = input("Enter an color: ")
if color in colors:
    index = colors.index(color)
    print(f"The index of {color} is {index}.")
```

```
Enter an color: blue
The index of blue is 2.
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

List

- Một số phương thức trong list

Method	Description
append(value)	Thêm value vào cuối danh sách
clear()	Xóa tất cả phần tử trong list
copy()	Trả về bản sao của list
count()	Trả về số phần tử của list
extend(iterable)	Thêm các phần tử của iterable vào cuối list
index(value)	Trả về vị trí tìm thấy value lần đầu tiên trong list, nếu không có báo lỗi

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

List

- Một số phương thức trong list

Phương thức	Mô tả
insert(index, value)	Chèn value vào vị trí index (index > len(list) → thêm vào cuối, index <0 →chèn trước index tính từ cuối list
pop([index])	Lấy phần tử tại vị trí index và xóa nó trong list, mặc định là phần tử cuối
remove(value)	Xóa phần tử value tìm thấy đầu tiên trong list
reverse()	Đảo thứ tự trong list

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

List

- Một số phương thức trong list
  - sort(key=None, reverse=False): sắp xếp danh sách theo thứ tự tăng
    - reverse=True: thứ tự dần
    - key: quy định tiêu chí sắp xếp

```
colors = ["yellow", "red", "Green", "Blue",  "white", "black"]
colors.sort(key=str.lower)
print(colors)
colors.sort(key=len)
print(colors)
items = [(1, "b"), (3, "a"), (2, "c")]
items.sort(key=lambda x: x[0])
print(items)
```

```
['black', 'Blue', 'Green', 'red', 'white', 'yellow']
['red', 'Blue', 'black', 'Green', 'white', 'yellow']
[(1, 'b'), (2, 'c'), (3, 'a')]
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

List

- Ví dụ

```
colors = ["red", "green", "blue", "yellow"]
print(colors)
#output: ['red', 'green', 'blue', 'yellow']
colors.append("orange")
print(colors) #output: ['red', 'green', 'blue', 'yellow', 'orange']
colors.insert(1, "purple")
print(colors) #output: ['red', 'purple', 'green', 'blue', 'yellow', 'orange']
colors.remove("yellow")
print(colors) #output: ['red', 'purple', 'green', 'blue', 'orange']
item = colors.pop()
print(item)#output: orange
print(colors)#output: ['red', 'purple', 'green', 'blue']
colors.clear()
print(colors)#output: []
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

List

- Ví dụ

```
colors = ["red", "green", "blue", "yellow"]
print(colors)
#output: ['red', 'green', 'blue', 'yellow']
print("{colors.count)}")
#output: 4
print(colors) #output: ['blue', 'green', 'red', 'yellow']
colors.sort()
print(colors) #output: ['blue', 'green', 'red', 'yellow']
colors.reverse()
print(colors)#output: ['yellow', 'red', 'green', 'blue']
points = (1, 4, 5, 9)
colors.extend(points)
print(colors) #output: ['yellow', 'red', 'green', 'blue', 1, 4, 5, 9]
```


TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

4.7 String

- String là đối tượng lưu trữ các ký tự liên tục nhau và không thể thay đổi khi chương trình thực thi.
- Chuỗi có thể đặt trong cặp nháy kép hoặc nháy đơn
- Dùng toán tử gán để tạo biến string
- Để tạo một biến string lưu nhiều dòng, đặt chuỗi ký tự trong ba dấu nháy kép hoặc nháy đơn

```
name = "Thanh"
address = """Phường Gò Vấp
Thành phố Hồ Chí Minh"""
print(name)
print(address)
```

```
Thanh
Phường Gò Vấp
Thành phố Hồ Chí Minh
```

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

String

- Các phương thức của string:

Method	Description
capitalize()	Converts the first character to upper case
casefold(), lower()	Converts string into lower case
title()	Converts the first character of each word to upper case
upper()	Converts a string into upper case
swapcase()	Swaps cases, lower case becomes upper case and vice versa
center(), ljust(), rjust()	Center, left, right justified

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

String

- Các phương thức của string:

Method	Description
count()	Returns the number of times a specified value occurs in a string
startswith()	Returns true if the string starts with the specified value
endswith()	Returns true if the string ends with the specified value
swapcase()	Swaps cases, lower case becomes upper case and vice versa
rstrip()	Returns a right trim version of the string
lstrip()	Returns a left trim version of the string

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

String

- Các phương thức của string:

Method	Description
index(), find()	Searches the string for a specified value and returns the position of where it was found
rindex(), rfind()	Searches the string for a specified value and returns the position of where it was found
replace()	Returns a string where a specified value is replaced with a specified value
split(), rsplit()	Splits the string at the specified separator, and returns a list
splitlines()	Splits the string at line breaks and returns a list

TRƯỜNG ĐẠI HỌC MỞ TP. HCM
Cơ hội học tập cho mọi người

String

- Các phương thức của string:

Method	Description
partition(), rpartition()	Returns a tuple where the string is parted into three parts
join()	Joins the elements of an iterable to the end of the string
format()	Formats specified values in a string
isalnum(), isalpha(), isdecimal(), isdigit(), isspace(), isnumeric(), islower(), isupper(),...	Check the characters, return True or False

Q/A?